MI VDEC API

Version 3.04

REVISION HISTORY

Revision No.	Description	Date
3.00	• Initial release	12/04/2020
3.01	• Added APIs for refreshing the repeated frame after paused decoding: • MI_VDEC_PauseChn, MI_VDEC_RefreshChn, MI_VDEC_ResumeChn	01/04/2021
3.02	• Added MI_VDEC_StepChn to pause the single frame playback function after decoding	02/24/2021

3.03	• Added Muffin decoding specifications
	• Added Muffin Muffin Data Flowchart
	• Added new APIs
	  • MI_VDEC_SetOutputPortAttrEx
	  • MI_VDEC_GetOutputPortAttrEx
	  • MI_VDEC_SetDestCropEx
	  • MI_VDEC_GetDestCropEx
		05/19/2021

3.04	• Transfer bDisableLowLatency in MI_VDEC_CreateDev to MI_VDEC_CreateChn, and support setting by chn • Added new parameters in MI_VDEC_CreateDev to decode the maximum width and height, which are used to adjust the maximum video stream resolution supported by the decoder • Added error type to MI_VDEC_GetChnStat	07/20/2021
	• Added PROCFS INTRODUCTION	08/25/2021

TABLE OF CONTENTS

1. OVERVIEW

1.1. Module Description

The video decoder module is used for fulfilling functions such as creating decoding channel, transmitting and controlling video streams, cropping and scaling output image, etc.

There are two main types of input sources for vdec module:

- User reads stream file and sends data to decoding module
- User sends the stream data received by the network directly to the decoding module

Table 1-1: Chip Decoding Specifications

Chip	Hardware Decoding Module	Maximum Number of Channels	Protocol	Resolution Range
Taiyaki	DEC2.0	16	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Takoyaki	DEC2.0	4	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Tiramisu	DEC2.0	16	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Mochi	DEC2.0	64x2	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128
Muffin	DEC2.0	64x2	H.264/H.265	H.264: max 4096x4096, min 176x128 H.265: max 4096x4096, min 176x128

1.2. Decoding Flowchart

1.2.1 Taiyaki Data Flowchart

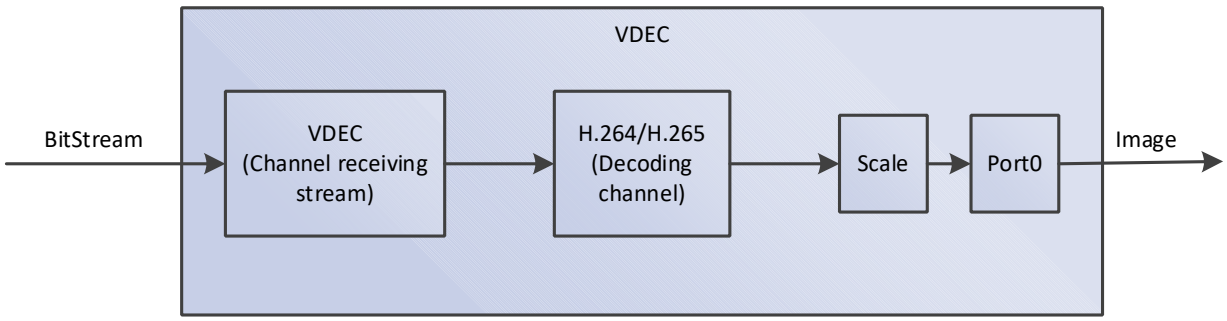


Figure 1-1: VDEC Data Flowchart

1.2.2 Takoyaki Data Flowchart

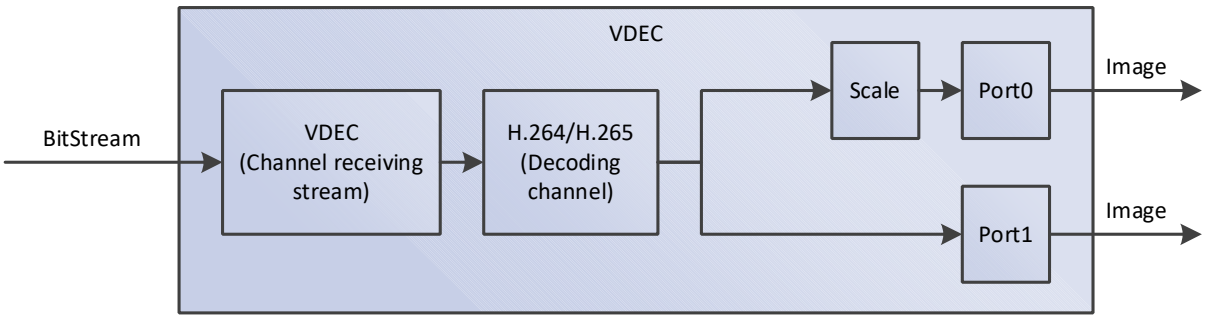


Figure 1-2: VDEC Data Flowchart

1.2.3 Tiramisu Data Flowchart

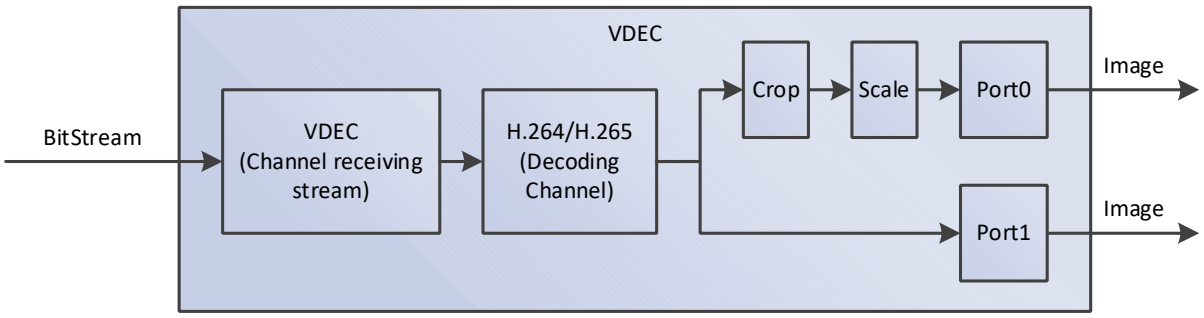


Figure 1-3: VDEC Data Flowchart

1.2.4 Muffin Data Flowchart

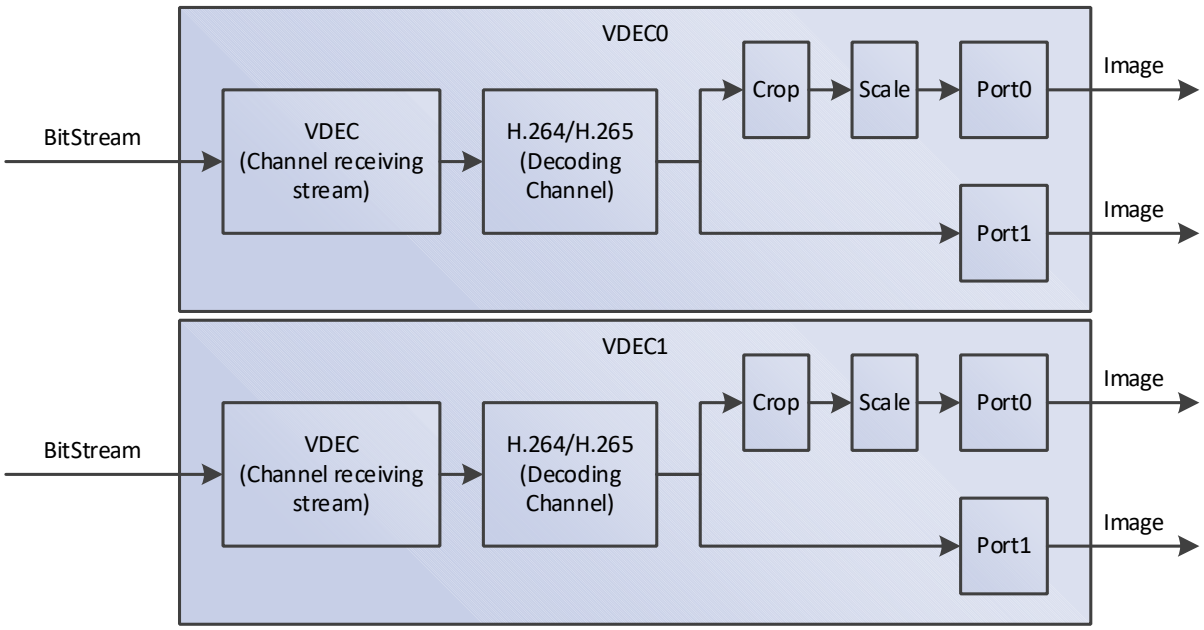


Figure 1-4 VDEC Data Flowchart

Muffin has two VDEC hardware with the same decoding capabilities, named Device0 and Device1.

1.2.5 Mochi Data Flowchart

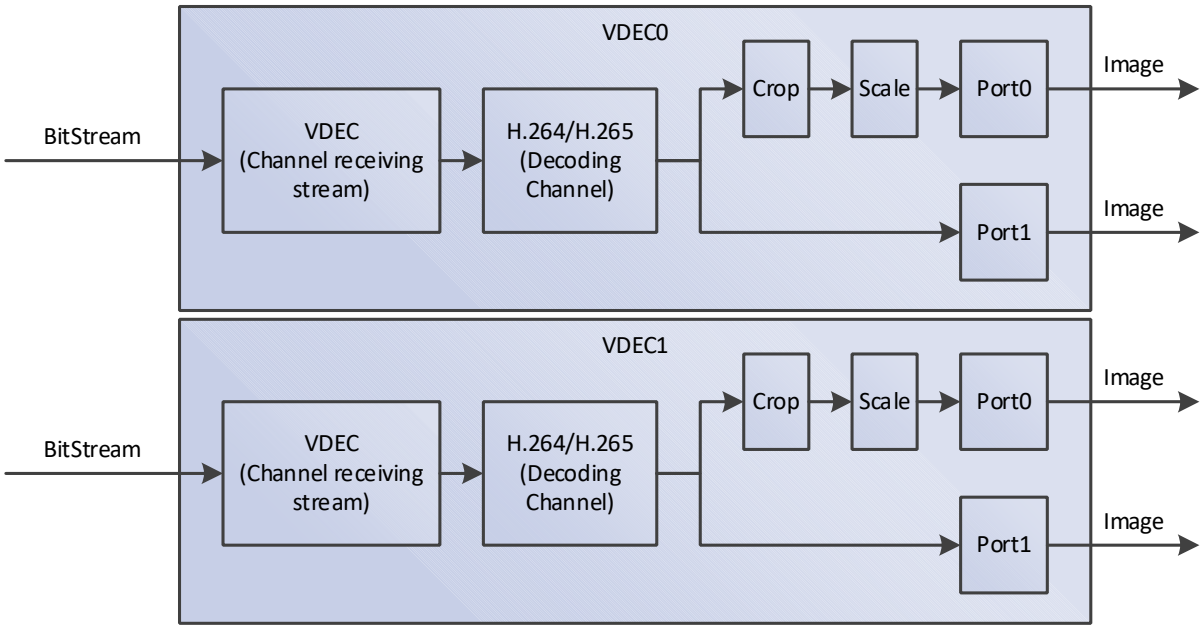


Figure 1-5 VDEC Data Flowchart

Mochi has two VDEC hardware with the same decoding capabilities, named Device0 and Device1.

1.3. Key Concepts

- Bitstream send mode

  VDEC decoder provides two ways to send bitstream:

  - Send by stream (E_MI_VDEC_VIDEO_MODE_STREAM): The user can send any length of bitstream to the decoder each time, and the decoding and splicing process of the bitstream frame is completed by the decoder. For H.264/H.265, the end of the current bitstream frame can only be confirmed after receiving the next bitstream. So, in this mode, the image will not be immediately output after inputting a frame of H.264/H.265 bitstream. Sending by stream is not supported.
  - Send by frame (E_MI_VDEC_VIDEO_MODE_FRAME): Each time the user sends a complete frame of video stream to the decoder and call once the send interface, the decoder considers that the frame coded stream has ended and begins to decode.

  Bitstream send mode can be set in the interface MI_VDEC_CreateChn.

- Image output order

  According to H.264/H.265 protocol, the decoded image may not be output immediately after decoding. VDEC decoder can output as soon as possible by setting different image output modes. Image output methods include the following two:

  - Decoding order: the decoded image is output in the order of decoding.
  - Display order: the decoded image is output according to the display order in the protocol.

  According to H.264/H.265 protocol, the decoding order of video may be inconsistent with the display order of decoded image. For example, when decoding B frame, P frame before and after B frame is needed as reference, so P frame after B frame is decoded before B frame, but B frame is output before P frame. Output in decoding order is a necessary condition for fast output. If the user chooses to output in decoding order, the decoding order and the display order of the video stream should be the same.

  The combination of sending video stream by frame and outputting by decoding sequence can achieve the purpose of fast decoding and output. The user must ensure that a complete frame video stream is sent

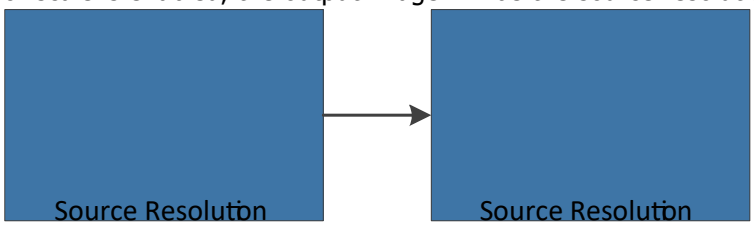
each time, and that the decoding sequence and display sequence of the video stream are the same. The image output mode will be adaptive to the display order, and the user setting is invalid.

- PTS
  When sending a bitstream in frame mode, the decoded output image time stamp PTS is the PTS sent by the user in the MI_VDEC_SendStream interface, and the decoder will not change this value. If the PTS value set by the user is -2, this image will not be displayed by the video display module (disp). If it is another value, it means that the decoder will not do anything.
  Note: PTS is not used for frame rate control. For frame rate control function, please refer to API description in MI_SYS.
- Bitstream buffer
  The bitstream buffer is used by the user to cache the coded stream input and send it to the decoder for decoding. There is no fixed formula for calculating the size of stream buffer.
  Generally, according to experience, if the resolution is less than or equal to D1, the recommended value is 512KB; if the resolution is in the range (d1, 1080p], the recommended value is 1MB; if the resolution is in the range (1080p,4K], the recommended value is 2MB. Users can choose reasonable value settings according to actual needs.
- eDpbBufMode
  By setting eDpbBufMode, users can set different buffer modes for decoding. According to different scenarios, different buffer modes can be selected to save memory. When E_MI_VDEC_DPB_MODE_NORMAL is set, memory cannot be saved. When E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF is set, it can only decode the stream that contains only one reference frame and only one DPB buffer is needed; when E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF is set, it can only decode the stream that contains only two reference frames and only two DPB buffer are needed.
- u32RefFrameNum
  u32reframenum represents the maximum number of reference frames. In the actual scene, the memory of the system may not be infinite. So the user can limit the number of reference frames according to the definition of the product, so as to prompt the user that the current video stream may have exceeded the specification. If the user sets eDpbBufMode to E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF or E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF, the parameter is invalid. If the user sets edpbbufmode to E_MI_VDEC_DPB_MODE_NORMAL and the number of reference frames is less than what the decoder actually needed, the decoded image may be abnormal or decoding timeout may happen.
- Output image scaling
  The user can call MI_VDEC_SetOutputPortAttr to scale the decoded image to output the image with the required resolution.
  Note that the decoder only supports scaling down the output image.
- Low latency
  The latency here refers to the latency generated by reordering the image queue to be displayed after decoding the B frame. When decoding the coded stream without B frame, the decoding order and the display order are the same, so there is no latency output, that is, the latency is low. When decoding bitstreams containing B frame, output latency will occur, because B frame has forward and backward reference, making it necessary to wait for the reference image to be decoded before the current frame can be decoded, and besides, the decoded image needs to be reordered.
  These two modes need to be completed by setting bDisableLowLatency. The default value is FALSE, and

the user does not need to set it. To decode bitstream with B frame, you need to set bDisableLowLatency to TRUE.

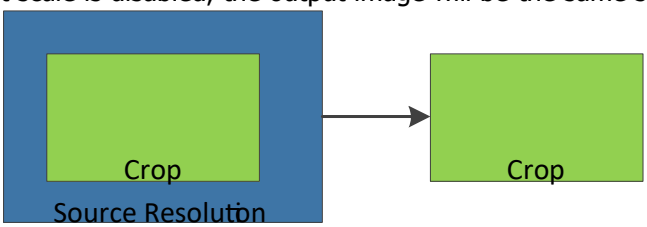
- Output buffer mode
  The user can call MI_VDEC_SetOutputPortLayoutMode to set the output buffer mode. This interface is used for Takoyaki, Tiramisu and Muffin. This interface can be called when the user needs to manually control the output buffer mode.
  If E_MI_VDEC_OUTBUF_LAYOUT_LINEAR is set, it indicates that the output buffer is in linear mode.
  If E_MI_VDEC_OUTBUF_LAYOUT_TILE is set, it indicates that the output buffer is in tile mode.
  If E_MI_VDEC_OUTBUF_LAYOUT_AUTO is set, it indicates that the output buffer will automatically switch between linear mode and tile mode.
- Output image cropping
  The user can call MI_VDEC_SetDestCrop to crop the output image. The coordinate X and the width must be aligned with 16. The coordinate Y and the height must be aligned with 2. If the cropping function and the scaling function are used at the same time, follow the processing sequence of cropping first and then scaling, and the cropped image size cannot be smaller than the scaled size, otherwise the interface will return the error code MI_ERR_VDEC_ILLEGAL_PARAM.The relationship between crop and scale is as shown below:

1. If neither crop nor scale is enabled, the output image will be the source resolution.

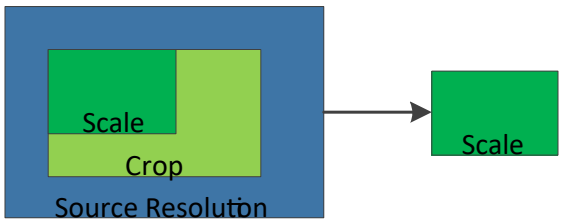


2. If crop is disabled but scale is enabled, the output image will be the same size as scaled info.

3. If crop is enabled but scale is disabled, the output image will be the same size as the cropped info.



4. If both crop and scale are enabled, the image will take the cropped size as a reference, then outputs a scaled image.



If enabling the scale, it will output the same size as the scale info.

5. Output image format

H264 and H265 decoding only support output NV12 image format.

2. API LIST

This module provides the following APIs:

Name of API	Function
MI_VDEC_CreateDev	Create decoding device
MI_VDEC_DestroyDev	Destroy decoding device
MI_VDEC_CreateChn	Create a decoding channel
MI_VDEC_DestroyChn	Destroy a decoding channel
MI_VDEC_GetChnAttr	Get decoding channel attribute
MI_VDEC_StartChn	Start the channel decoding
MI_VDEC_StopChn	Stop the channel decoding
MI_VDEC_GetChnStat	Get decoding channel status
MI_VDEC_FlushChn	Clear decoding channel cache
MI_VDEC_ResetChn	Reset decoding channel
MI_VDEC_SetChnParam	Set decoding channel parameter
MI_VDEC_GetChnParam	Get decoding channel parameter
MI_VDEC_SendStream	Send video stream data to decoding channel
MI_VDEC_PauseChn	Pause channel decoding
MI_VDEC_RefreshChn	Refresh the channel and re-decode the current frame
MI_VDEC_ResumeChn	Resume channel decoding
MI_VDEC_StepChn	Channel single frame decoding
MI_VDEC_GetUserData	Get decoding channel user data
MI_VDEC_ReleaseUserData	Release decoding channel user data
MI_VDEC_SetDisplayMode	Set decoding channel display mode
MI_VDEC_GetDisplayMode	Get decoding channel display mode
MI_VDEC_SetOutputPortAttr	Set decoding channel output port attribute
MI_VDEC_GetOutputPortAttr	Get decoding channel output port attribute
MI_VDEC_SetOutputPortLayoutMode	Set output port layout mode
MI_VDEC_GetOutputPortLayoutMode	Get output port layout mode
MI_VDEC_SetDestCrop	Set the decoded image cropping attribute
MI_VDEC_GetDestCrop	Get the decoded image cropping attribute

Name of API	Function
MI_VDEC_SetChnErrHandlePolicy	Set the output strategy of the decoded channel error MB data frame

2.1. MI_VDEC_CreateDev

- Function

  Create decoding device.

- Syntax

  MI_S32 MI_VDEC_CreateDev(MI_VDEC_DEV VdecDev, MI_VDEC_InitParam_t *pstInitParam);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
pstInitParam	Decoding initialization parameters pointer. Data type: MI_VDEC_InitParam_t	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INITED	Device to be initialized is already initialized.
MI_ERR_VDEC_NULL_PTR	Null pointer
MI_ERR_VDEC_NOT_CONFIG	Not configured before use
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameters or input parameters exceed the channel decoding capability.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - The device number cannot exceed the maximum range.
  - The interface is different in the use of different platforms, see the following table for details.

Chips/Platforms	Interface usage description
Taiyaki	Without this interface, settings are not supported.
Takoyaki	Optional call.
Tiramisu	Optional call.
Muffin	Must be called.

- If you choose to use this interface, you must call it before creating a decoding channel; otherwise, it will return the error code MI_ERR_VDEC_INITED which means the device has been initialized.
- If the interface is called repeatedly, the error code MI_ERR_VDEC_INITED will be returned.

➢ Example

```
MI_S32 StartVdec(void)
{
   MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
     MI_VDEC_DEV VdecDev = 0;
   MI_VDEC_CHN VdecChn = 0;
   MI_VDEC_InitParam_t stInitParam;
   MI_VDEC_ChnAttr_t stChnAttr;

   memset(&stInitParam, 0x0, sizeof(MI_VDEC_InitParam_t));
   memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

   s32Ret = MI_VDEC_CreateDev(VdecDev, &stInitParam);
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_CreateDev failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }

   VdecChn = 0;
   stChnAttr.eCodecType    = E_MI_VDEC_CODEC_TYPE_H264;
   stChnAttr.u32PicWidth   = 1920;
   stChnAttr.u32PicHeight  = 1080;
   stChnAttr.eVideoMode    = E_MI_VDEC_VIDEO_MODE_FRAME;
   stChnAttr.u32BufSize    = 1024*1024;
   stChnAttr.eDpbBufMode   = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
   stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
    stChnAttr.u32Priority = 0;
    stChnAttr.stVdecVideoAttr.stErrHandlePolicy.bUseCusPolicy = FALSE;
    stChnAttr.stVdecVideoAttr.bDisableLowLatency = FALSE;

   s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }
```

```
    return MI_SUCCESS;
}
```

- Related API

  N/A.

2.2. MI_VDEC_DestroyDev

- Function

  Destroy decoding device.

- Syntax

  MI_S32 MI_VDEC_DestroyDev (MI_VDEC_DEV VdecDev);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_NOT_INIT	Device is not initialized.
MI_ERR_VDEC_NOT_CONFIG	Not configured before use

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - The use of this interface varies with different platforms, please refer to the note of MI_VDEC_CreateDev interface for details.
  - The device number cannot exceed the maximum range.
  - If you choose to call this interface, you have to call it after destroying all the decoding channels; otherwise, the interface will destroy all the decoding channels automatically.
  - If the interface is called repeatedly, the error code MI_ERR_VDEC_NOT_INIT will be returned.

- Example

```
MI_S32 StopVdec(void)
{
```

```
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
  MI_VDEC_CHN VdecChn = 0;

  s32Ret = MI_VDEC_DestroyChn(VdecDev , VdecChn);
  if(MI_SUCCESS != s32Ret)
  {
     printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
     return s32Ret;
  }

  //Confirm that you have destroyed all channels
  s32Ret = MI_VDEC_DestroyDev(VdecDev);
  if(MI_SUCCESS != s32Ret)
  {
     printf("MI_VDEC_DestroyDev failed, s32Ret: 0x%x.\n", s32Ret);
     return s32Ret;
  }

  return MI_SUCCESS;
}
```

- Related API

  N/A.

2.3. MI_VDEC_CreateChn

- Function

  Create a video decoding channel

- Syntax

  MI_S32 MI_VDEC_CreateChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,
  MI_VDEC_ChnAttr_t
  *pstChnAttr)

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Decoding channel number Range: [0, MI_VDEC_MAX_CHN_NUM)	Input
pstChnAttr	Decoding channel attribute pointer Data type: MI_VDEC_ChnAttr_t	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid Channel ID. Range: [0, MI_VDEC_MAX_CHN_NUM).
MI_ERR_VDEC_NOT_INIT	Module not successfully initialized
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter or inputted parameter exceeding channel decoding capability.
MI_ERR_VDEC_CHN_EXIST	Channel to be created already exists.
MI_ERR_VDEC_NOMEM	Memory allocation fails (due to for example insufficient memory).
MI_ERR_VDEC_NULL_PTR	Null pointer
MI_ERR_VDEC_NOT_SUPPORT	This operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - The channel number cannot exceed the maximum channel range allowed.
  - Before creating a decoding channel, you need to ensure that the decoding channel has not been created or destroyed, otherwise the error code MI_ERR_VDEC_CHN_EXIST will be returned.
  - If eDpbBufMode is set to E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF or E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF, we only guarantee that the decoder can decode the bitstream encoded by Sstar encoder well, but the third party's decode bitstream is not guaranteed to be fully supported. In addition, eDpbBufMode should be consistent with reference frame number of the bitstream.
  - If you need to decode the B-frame code stream, you need to set bDisableLowLatency to TRUE to ensure that the decoder outputs the decoded images in the display order; otherwise, the output images will have jitter and inconsistency problems.

- Example

```
MI_S32 StartVdec(void)
{
MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
   MI_VDEC_DEV VdecDev = 0;
   MI_VDEC_CHN VdecChn = 0;
   MI_VDEC_DisplayMode_e eDisplayMode = E_MI_VDEC_DISPLAY_MODE_MAX;
   MI_VDEC_ChnAttr_t stChnAttr;
   MI_VDEC_OutputPortAttr_t stOutputPortAttr;
```

```
    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));
    memset(&stOutputPortAttr, 0x0, sizeof(MI_VDEC_OutputPortAttr_t));

    VdecChn = 0;
    stChnAttr.eCodecType    = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth   = 1920;
    stChnAttr.u32PicHeight  = 1080;
    stChnAttr.eVideoMode    = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize    = 1024*1024;
    stChnAttr.eDpbBufMode   = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
stChnAttr.u32Priority = 0;
stChnAttr.stVdecVideoAttr.stErrHandlePolicy.bUseCusPolicy = FALSE;
stChnAttr.stVdecVideoAttr.bDisableLowLatency = FALSE;

    s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    eDisplayMode = E_MI_VDEC_DISPLAY_MODE_PLAYBACK;
    s32Ret = MI_VDEC_SetDisplayMode (VdecDev, VdecChn, eDisplayMode);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SetDisplayMode failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    s32Ret = MI_VDEC_StartChn(VdecDev, VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_StartChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    stOutputPortAttr.u16Width = 640;
    stOutputPortAttr.u16Height = 480;
    s32Ret = MI_VDEC_SetOutputPortAttr(VdecDev, VdecChn, &stOutputPortAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SetOutputPortAttr failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
```

```
}

stCropCfg.bEnable   = TRUE;
stCropCfg.stRect.u16X  = 0;
stCropCfg.stRect.u16Y  = 0;
stCropCfg.stRect.u16Width = 720;
stCropCfg.stRect.u16Height = 576;
s32Ret = MI_S32 MI_VDEC_SetDestCrop(VdecDev, VdecChn, &stCropCfg);
if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_SetDestCrop failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

    return MI_SUCCESS;
}
```

- Related API

  N/A.

2.4. MI_VDEC_DestroyChn

- Function

  Destroy a video decoding channel.

- Syntax

  MI_S32 MI_VDEC_DestroyChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_NOT_STOP	Decoding is not stopped before destroying the channel.
MI_ERR_VDEC_CHN_UNEXIST	Channel does not exist.

MI_ERR_VDEC_NOT_SUPPORT	This operation or function is not supported.
MI_ERR_VDEC_NULL_PTR	The parameter is a null pointer.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before destroying the decoding channel, you must ensure that the channel has been created, otherwise the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.
  - Before destroying the decoding channel, decoding should be stopped first, otherwise the error code MI_ERR_VDEC_CHN_NOT_STOP will be returned.

- Example

```
MI_S32 StopVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
   MI_VDEC_CHN VdecChn = 0;

   // Destroy send stream thread
   ...

   s32Ret = MI_VDEC_StopChn(VdecDev, VdecChn);
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_StopChn failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }

   s32Ret = MI_VDEC_DestroyChn(VdecDev, VdecChn);
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }

   return MI_SUCCESS;
}
```

- Related API

  N/A.

2.5. MI_VDEC_StartChn

- Function

  Start the channel decoding.

- Syntax

  MI_S32 MI_VDEC_StartChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_CHN_NOT_STOP	The channel does not stop decoding.
MI_ERR_VDEC_NULL_PTR	The parameter is a null pointer.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before the channel starts decoding, be sure the channel has been created; otherwise, the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.
  - After the channel starts decoding, MI_VDEC_SendStream will be called to send stream successfully.
  - Repeated calling of the interface will return MI_ERR_VDEC_CHN_NOT_STOP.

- Example

  See the example of MI_VDEC_CreateChn.

- Related API

  N/A.

2.6. MI_VDEC_StopChn

- Function

Stop the channel decoding.

- Syntax

  MI_S32 MI_VDEC_Stop Chn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_FAILED	Unsuccessful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_NULL_PTR	The parameter is a null pointer.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_CHN_NOT_START	Channel not started or already stopped
MI_ERR_VDEC_BUSY	System is busy.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this interface, be sure the channel has been created; otherwise, the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.
  - Before calling this interface, be sure the channel has been enable; otherwise, the error code MI_ERR_VDEC_CHN_NOT_START will be returned.
  - Calling the video stream sending interface MI_VDEC_SendStream after calling this function will return failed.
  - Repeated calling of this interface will return the error code MI_ERR_VDEC_CHN_NOT_START.
  - When decoding B frames, if the MI_DISP module is bound to the back stage of MI_VDEC, please exit the MI_DISP module before MI_VDEC stops the channel; otherwise, the linear buffer will be released directly when MI_VDEC exits, but MI_DISP is still accessing the output buffer of MI_VDEC at this time, which will cause some Memory problem; if there is a buffer that has not been returned when MI_VDEC exits, the error code MI_ERR_VDEC_BUSY will be reported.

- Example

See the example of MI_VDEC_DestroyChn.

- Related API

N/A.

2.7. MI_VDEC_GetChnAttr

- Function

Get video decoding channel attribute.

- Syntax

MI_S32 MI_VDEC_GetChnAttr(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_ChnAttr_t *pstChnAttr);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstChnAttr	Decoding channel attribute pointer. Data type: MI_VDEC_ChnAttr_t.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_NULL_PTR	The input parameter is a null pointer.
MI_ERR_VDEC_CHN_UNEXIST	Channel does not exist.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before getting channel attribute, be sure the channel has been created, otherwise the error code MI_ERR_VDEC_CHN_UNEXIST will be return.

- Example

N/A.

- Related API

N/A.

2.8. MI_VDEC_GetChnStat

- Function

  Get the decoding channel status.

- Syntax

  MI_S32 MI_VDEC_GetChnStat (MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_ChnStat_t *pstChnStat);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstChnStat	Decoding channel status structure pointer.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_NULL_PTR	The input parameter is a null pointer.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before getting channel status, be sure the channel has been created, otherwise the error code MI_ERR_VDEC_CHN_UNEXIST will be return.

- Example

  See the example of MI_VDEC_SendStream.

- Related API

  N/A.

2.9. MI_VDEC_FlushChn

- Function

  Clear decoding channel cache.

- Syntax

  MI_S32 MI_VDEC_FlushChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_CHN_NOT_START	Channel not started or already stopped.
MI_ERR_VDEC_NULL_PTR	The parameter is a null pointer.
MI_ERR_VDEC_BUSY	The decoder is BUSY

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - This interface can be used to clear the cached data of decoding channel when switching to other GOP during decoding, so that decoding can continue.

- Example

  N/A.

- Related API

  N/A.

2.10. MI_VDEC_ResetChn

- Function

  Reset a decoding channel.

- Syntax

  MI_S32 MI_VDEC_ResetChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output

VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_NOT_PERM	Unable to reset channel. This function cannot be called along with the MI_VDEC_SendStream.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - This interface is currently not supported.

- Example

  N/A.

- Related API

  N/A.

2.11. MI_VDEC_SetChnParam

- Function

  Set decoding channel parameter.

- Syntax

  MI_S32 MI_VDEC_SetChnParam(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_ChnParam_t* pstChnParam);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

pstChnParam	Channel parameter.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_NOT_PERM	Illegal operation. Before calling this function, call MI_VDEC_StopChn to stop channel decoding first.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - This interface is currently not supported.

- Example

  N/A.

- Related API

  N/A.

2.12. MI_VDEC_GetChnParam

- Function

  Get decoding channel parameter.

- Syntax

  MI_S32 MI_VDEC_GetChnParam(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_ChnParam_t* pstChnParam);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstChnParam	Channel parameter.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - This interface is currently not supported.

- Example

  N/A.

- Related API

  N/A.

2.13. MI_VDEC_SendStream

- Function

  Send video stream data to decoding channel.

- Syntax

  MI_S32 MI_VDEC_SendStream(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_VideoStream_t *pstVideoStream, MI_S32 s32MilliSec);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstVideoStream	Decoding video stream data pointer. Please refer to the definition given in MI_VDEC_VideoStream_t.	Input
s32MilliSec	Set data push timeout time parameter. Range: -1: Blocked. 0: Not blocked. Positive values: Timeout time, unit is ms.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_FAILED	Unsuccessful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_NOT_START	The channel is disabled
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_BUF_FULL	The stream buffer before decoding is full.
MI_ERR_VDEC_NOMEM	Memory allocation fails (due to for example insufficient memory).
MI_ERR_VDEC_NOT_CONFIG	Not configured before use.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Stream-based data transmissions is currently not supported.
  - For frame-based case, an entire data frame should be transmitted for each transmission at a time.
  - If the user sets pstVideoStream->u64PTS = -2 when sending the stream, VDEC will discard the frame after decoding. It can be set as required.

- Example

An entire data frame must be sent for each transmission in frame-based transmission. If the current data frame failed during the transmission, re-transmission is required.

```
MI_S32 VdecSendStream(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
  MI_VDEC_CHN VdecChn = 0;
  MI_S32 s32MilliSec = 0;
    MI_VDEC_VideoStream_t stVideoStream;
    MI_VDEC_ChnAttr_t stChnAttr;
  MI_VDEC_ChnStat_t stChnStat;

  do{
     //Check if you need stop sending stream
     if(bStop)
     {
        break;
     }
```

```
    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));
    s32Ret = MI_VDEC_GetChnAttr(VdecDev, VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
       printf("MI_VDEC_GetChnAttr failed, s32Ret: 0x%x.\n", s32Ret);
       return s32Ret;
    }

    memset(&stChnStat, 0x0, sizeof(MI_VDEC_ChnStat_t));
    s32Ret = MI_VDEC_GetChnStat(VdecDev, VdecChn, &stChnStat);
    if(MI_SUCCESS != s32Ret)
    {
       printf("MI_VDEC_GetChnStat failed, s32Ret: 0x%x.\n", s32Ret);
       return s32Ret;
    }

    //suggest to check chn status
    if(stChnAttr.u32Bufsize - stChnStat.u32LeftStreamBytes < u32StreamSize)
    {
       continue;
    }

    memset(&stVideoStream, 0x0, sizeof(MI_VDEC_VideoStream_t));
    stVideoStream.pu8Addr = pu8StreamBuf;
    stVideoStream.u32Len  = u32StreamSize;
    stVideoStream.u64PTS  = u64StreamPts;
    stVideoStream.bEndOfFrame  = TRUE;
    stVideoStream.bEndOfStream = FALSE;
    s32MilliSec = 0;                  //0ms
    s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVideoStream, s32MilliSec);
    if(MI_SUCCESS != s32Ret)
    {
       printf("MI_VDEC_SendStream failed, s32Ret: 0x%x.\n", s32Ret);
       continue;
    }
  }while(!ShouldStop);

  return MI_SUCCESS;
}
```

- Related API

  N/A.

2.14. MI_VDEC_PauseChn

- Function

  Pause channel decoding.

- Syntax

  MI_S32 MI_VDEC_PauseChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_CHN_NOT_START	The channel is not started or disabled.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - After calling this interface, the decoder stops decoding; at this time, the upper layer can continue to call the MI_VDEC_SendStream interface to send streams, but when the stream buffer is full, MI_VDEC_SendStream will return MI_ERR_VDEC_BUF_FULL.
  - This interface can be called repeatedly without error.

- Example

```
MI_S32 PauseChn(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev  = 0;
    MI_VDEC_CHN VdecChn  = 0;
    MI_U32 u32RefreshCnt = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
```

```
    //refresh 10 times
    while (u32RefreshCnt < 10)
    {
        s32Ret = MI_VDEC_RefreshChn(VdecDev, VdecChn);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_RefreshChn failed, s32Ret: 0x%x.\n", s32Ret);
            break;
        }
        u32RefreshCnt ++;
        usleep(100*1000);
    }

    s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    return MI_SUCCESS;
}
```

- Related API

  N/A.

2.15. MI_VDEC_RefreshChn

- Function

  Refresh the channel and re-decode the current frame.

- Syntax

  MI_S32 MI_VDEC_RefreshChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful

Return Value	Description
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_CHN_NOT_START	The channel is not started or disabled.
MI_ERR_VDEC_NOT_DISABLE	The channel is not disabled.
MI_ERR_VDEC_BUF_EMPTY	The stream buffer is empty.
MI_ERR_VDEC_CHN_NO_CONTENT	The stream buffer content is invalid.
MI_ERR_VDEC_BUSY	The decoder is BUSY and cannot refresh channel temporarily.

➢ Requirement

- Header: mi_vdec.h
- Library: libmi_vdec.a/libmi_vdec.so

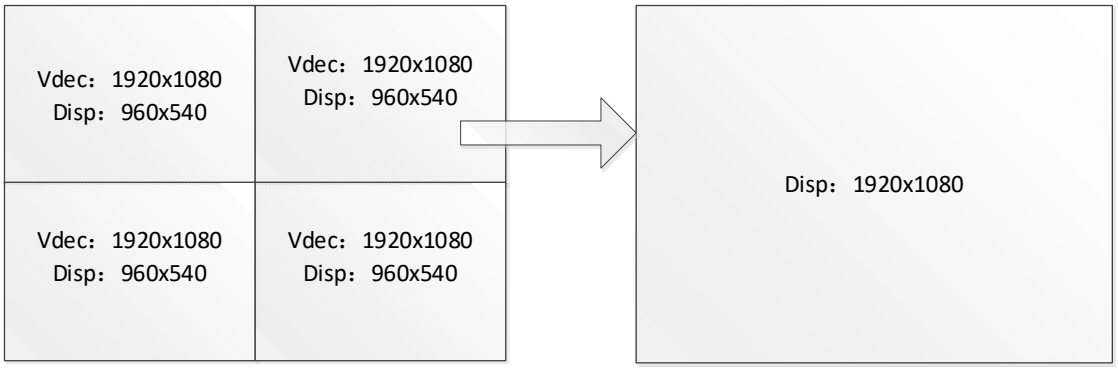
➢ Note

- When calling this interface, please ensure that MI_VDEC_PauseChn has been called to stop decoding, otherwise it will return MI_ERR_VDEC_NOT_DISABLE.
- Every time this interface is called, it will trigger the decoder to re-decode the current frame; it can be called more than once, and the zoom factor can be adjusted with MI_VDEC_SetOutputPortAttr to adapt to different scene requirements.
- When calling this interface, please ensure that there is still data to be decoded in the video stream buffer, otherwise refresh cannot be completed.
- When the user calls the interface continuously, if the last refresh or step task is not completed, the interface will return MI_ERR_VDEC_BUSY.
- When decoding the B frame, this function is not supported.

➢ Example

Refer the example of MI_VDEC_PauseChn and MI_VDEC_StepChn.

- At present, only Tiramisu and Muffin series support digital zoom and picture capture during pause. Example: Suppose that the output resolution of MI_DISP in preview mode is 1920x1080, and the 4 channels are all 1080P source streams while decoding and previewing, the output resolution of each channel is set to 960x540 for output display. If you pause the decoding and switch channel 2 to full-screen preview, it will be displayed in full-screen 1920x1080, and the image data will be distorted. By calling MI_VDEC_RefreshChn, the 1920x1080 image of the current pause data frame can be displayed without distortion.



The channel relies on the combination of MI_VDEC_PauseChn, MI_VDEC_RefreshChn, MI_VDEC_ResumeChn and MI_VDEC_SendStream to realize the pause electronic zoom function, so that images with different resolutions can be obtained for the same bitstream data frame. As follows, image 3 needs to be output 4 times repeatedly.

Calling sequence of API combination:

Step 1: Call MI_VDEC_SendStream to send the video stream data of data frame '1' and '2'.

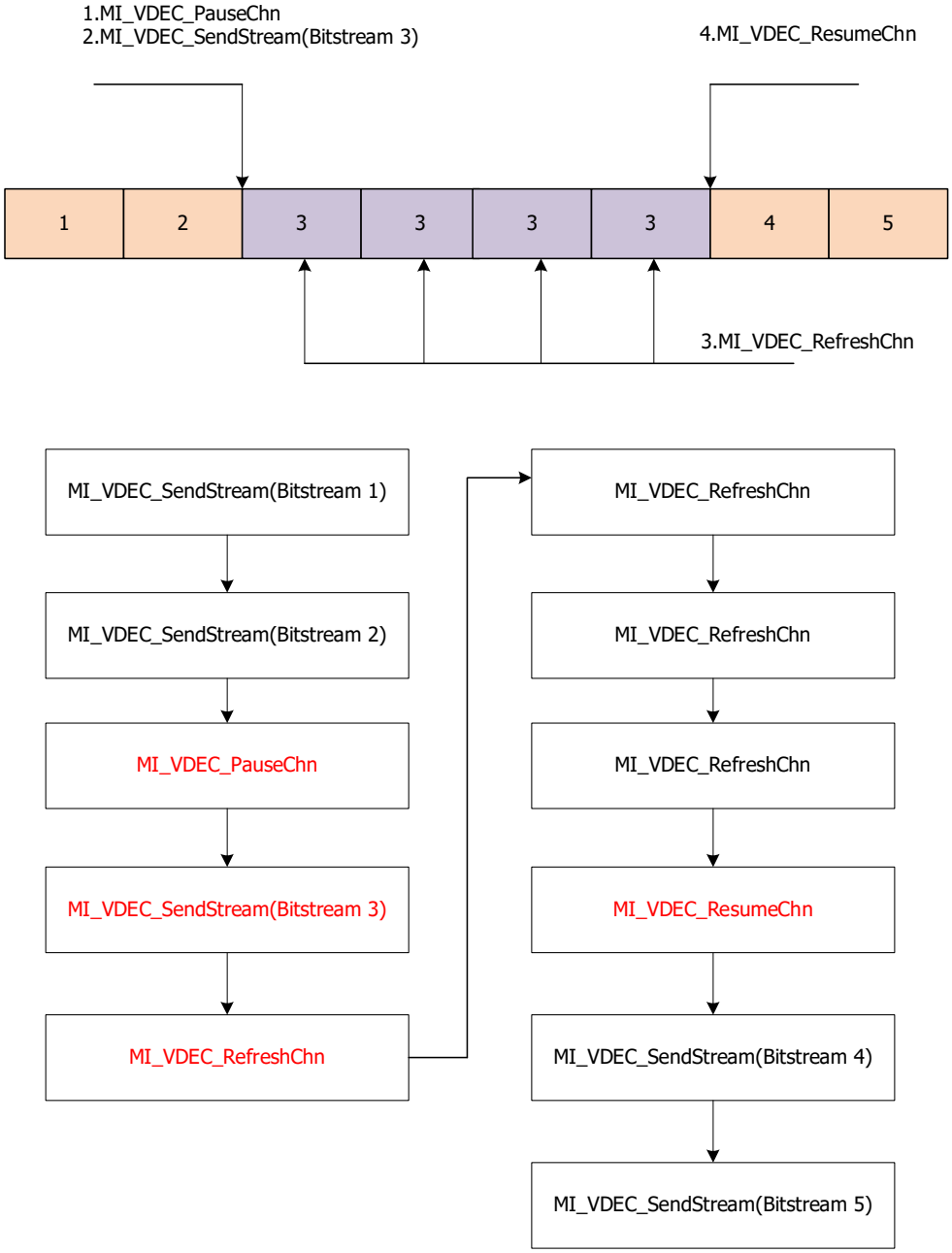
Step 2: Call MI_VDEC_PauseChn to pause the decoder and enter the decoded output repeat image frame mode.

Step 3: Call MI_VDEC_SendStream to send the video stream data that needs to repeatedly output image "3" to the decoder.

Step 4: Repeatedly call MI_VDEC_RefreshChn to output image '3', and output 1 frame of image for each call. The number of calls is unlimited. The current example is called 4 times.

Step 5: Call MI_VDEC_ResumeChn to exit the repeated decoding same image frame mode.

Step 6: Call MI_VDEC_SendStream to send the video stream data of data frame '4' and '5'.



- Related API

  N/A.

2.16. MI_VDEC_ResumeChn

- Function

  Resume channel decoding.

- Syntax

  MI_S32 MI_VDEC_ResumeChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_CHN_NOT_START	The channel is not started or disabled.
MI_ERR_VDEC_BUSY	The decoder status is BUSY and cannot resume channel.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this interface, be sure the channel has been created; otherwise, the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.
  - Before calling this interface, be sure the channel has been enable; otherwise, the error code MI_ERR_VDEC_CHN_NOT_START will be returned.
  - It is allowed to call this interface repeatedly.
  - If the last refresh or step task is not completed when calling this interface, the interface will return MI_ERR_VDEC_BUSY.

- Example

  Refer the example of MI_VDEC_PauseChn and MI_VDEC_StepChn.

- Related API

  N/A.

2.17. MI_VDEC_StepChn

- Function

  Channel single frame decoding.

- Syntax

  MI_S32 MI_VDEC_StepChn(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_NOT_STARTED	The channel is not started or disabled.
MI_ERR_VDEC_BUF_EMPTY	The stream buffer is empty.
MI_ERR_VDEC_CHN_NO_CONTENT	The stream buffer content is invalid.
MI_ERR_VDEC_BUSY	The decoder is busy and cannot step channel temporarily.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Each time this interface is called, it will trigger the decoder to decode the next frame; it can be called multiple times, and the zoom factor can be adjusted with MI_VDEC_SetOutputPortAttr to suit different scene requirements.
  - When calling this interface, the user needs to ensure that there is still data to be decoded in the video stream buffer, otherwise the step cannot be completed.
  - When the user calls the interface multiple times, if the last refresh or step task is not completed, the error code MI_ERR_VDEC_BUSY will be returned.

- Example

```
Case 1:
MI_S32 StepChn(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev  = 0;
    MI_VDEC_CHN VdecChn  = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
```

```
        printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
      }

      /// It can be replaced by another independent thread calling MI_VDEC_SendStream
    s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ SendStream failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
      }

    s32Ret = MI_VDEC_StepChn(VdecDev, VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_StepChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    usleep(20*1000);

    s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    return MI_SUCCESS;
}

Case 2:
MI_S32 refresh_and_step_chn(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev  = 0;
MI_VDEC_CHN VdecChn  = 0;
MI_U32 u32LoopCnt = 0;

    //While decoding...

    s32Ret = MI_VDEC_PauseChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_PauseChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
```

```
    }

    /// It can be replaced by another independent thread calling MI_VDEC_SendStream
    s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ SendStream failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    /*Loop 10 times*/
    While(u32LoopCnt < 10)
{
    s32Ret = MI_VDEC_RefreshChn(VdecDev, VdecChn);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_RefreshChn failed, s32Ret: 0x%x.\n", s32Ret);
            return s32Ret;
        }
        usleep(20*1000);

        /// It can be replaced by another independent thread calling MI_VDEC_SendStream
        s32Ret = MI_VDEC_SendStream(VdecDev, VdecChn, &stVdecStream, 0);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_ SendStream failed, s32Ret: 0x%x.\n", s32Ret);
            return s32Ret;
        }

        s32Ret = MI_VDEC_StepChn(VdecDev, VdecChn);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_StepChn failed, s32Ret: 0x%x.\n", s32Ret);
            return s32Ret;
        }
        usleep(20*1000);
        u32LoopCnt ++;
    }

    s32Ret = MI_VDEC_ResumeChn(VdecDev, VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_ResumeChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
```

```
    }
    return MI_SUCCESS;
}
```

- Related API

  N/A.

2.18. MI_VDEC_GetUserData

- Function

  Get decoding channel user data.

- Syntax

  MI_S32 MI_VDEC_GetUserData(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_UserData_t *pstUserData, MI_S32 s32MilliSec);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
s32MilliSec	Get user data timeout definition. Range: -1: Blocked. 0: Not blocked. Positive values: Timeout time, unit is ms.	Input
pstUserData	Decoded user data. Please refer to the definition given in MI_VDEC_UserData_t.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_NOT_PERM	Illegal operation, e.g. channel not ready.
MI_ERR_VDEC_BUF_FULL	Data frame buffer full after decoding.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - This interface is currently not supported.
- Example

  N/A.
- Related API

  N/A.

2.19. MI_VDEC_ReleaseUserData

- Function

  Release user data.
- Syntax

  MI_S32 MI_VDEC_ReleaseUserData(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_UserData_t * pstUserData);
- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstUserData	Decoded user data pointer, gotten by MI_VDEC_GetUserData interface.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_NOT_PERM	Illegal operation, e.g. channel not ready.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so
- Note
  - This interface is currently not supported.
- Example

  N/A.

- Related API

  N/A.

2.20. MI_VDEC_SetDisplayMode

- Function

  Set display mode.

- Syntax

  MI_S32 MI_VDEC_SetDisplayMode(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_DisplayMode_e eDisplayMode);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
eDisplayMode	Display mode enumeration. Please refer to the definition given in MI_VDEC_DisplayMode_e.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before setting display mode, be sure the channel has been created; otherwise, the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.
  - After optimizing the frame rate control mechanism, MI_VDEC does not need to lose frames.
    Therefore, the current setting of the interface is invalid.
  - Preview mode: In order to maintain the real-time preview of the decoded image, MI_VDEC will drop the decoded image immediately when the bitstream buffer accumulate to a certain extent. Thus, it can quickly clean up the accumulated data in the bitstream

buffer and
ensure that the bitstream received in real time is decoded immediately.

- Playback mode: In order to maintain the coherence of the decoded image, MI_VDEC will not drop the decoded image even if the bitstream buffer is stacked.Then, it can avoid displaying images that are stuck and incoherent.

➢ Example

See the example of MI_VDEC_CreateChn.

➢ Related API

N/A.

2.21. MI_VDEC_GetDisplayMode

➢ Function

Get display mode.

➢ Syntax

MI_S32 MI_VDEC_GetDisplayMode(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_DisplayMode_e *peDisplayMode);

➢ Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
peDisplayMode	Display mode enumeration pointer. Please refer to the definition given in MI_VDEC_DisplayMode_e.	Input

➢ Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_NULL_PTR	The input parameter is a null pointer.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.

➢ Requirement

- Header: mi_vdec.h
- Library: libmi_vdec.a/libmi_vdec.so

➢ Note

N/A.

- Example

  See the example of MI_VDEC_DestroyChn.

- Related API

  N/A.

2.22. MI_VDEC_SetOutputPortAttr

- Function

  Set decoding channel output port attribute

- Syntax

  MI_S32 MI_VDEC_SetOutputPortAttr (MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstOutputPortAttr	Output port attribute. Please refer to the definition given in MI_VDEC_OutputPortAttr_t.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_NULL_PTR	Null pointer.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the channel to be used has been created; otherwise, an error code of channel not created (MI_ERR_VDEC_CHN_UNEXIST) will be returned.
  - The scaling width and height need to be aligned by 2.

- Scaling range [1/8, 1].
- Scaling up is not supported.
- If MI_VDEC_SetDestCrop and MI_VDEC_SetOutputPortAttr are calling together, make sure that the cropping size is not smaller than the scaling size. Otherwise, an error code of illegal parameter (MI_ERR_VDEC_ILLEGAL_PARAM) will be returned.

➢ Example

See the example of MI_VDEC_CreateChn.

➢ Related API

N/A.

2.23. MI_VDEC_GetOutputPortAttr

➢ Function

Get decoding channel output port attribute.

➢ Syntax

MI_S32 MI_VDEC_GetOutputPortAttr(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);

➢ Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstOutputPortAttr	Output port attribute. Please refer to the definition given in MI_VDEC_OutputPortAttr_t	Output

➢ Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed
MI_ERR_VDEC_NULL_PTR	Null pointer.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

➢ Requirement

- Header: mi_vdec.h
- Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the channel to be used has been created; otherwise, an error code of channel not created (MI_ERR_VDEC_CHN_UNEXIST) will be returned.

- Example

  See the example of MI_VDEC_DestroyChn.

- Related API

  N/A.

2.24. MI_VDEC_SetOutputPortLayoutMode

- Function

  Set output port layout mode

- Syntax

  MI_S32 MI_VDEC_SetOutputPortLayoutMode(MI_VDEC_DEV VdecDev, MI_VDEC_OutbufLayoutMode_e eBufTileMode);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
eBufTileMode	Output buffer mode. Please refer to the definition given in MI_VDEC_OutbufLayoutMode_e.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_FAILED	Unsuccessful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_NOT_INIT	Device not initialized.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_BUSY	System is busy.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the device has been initialized; otherwise, the error code MI_ERR_VDEC_NOT_INIT will be returned.
  - This interface should be called before decoding, otherwise the error code MI_ERR_VDEC_BUSY will be returned.

- Example

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_DEV VdecDev = 0;
   MI_VDEC_CHN VdecChn = 0;
   MI_VDEC_OutbufLayoutMode_e eBufTileMode = E_MI_VDEC_OUTBUF_LAYOUT_MAX;
   MI_VDEC_ChnAttr_t stChnAttr;

   memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

   eBufTileMode = E_MI_VDEC_OUTBUF_LAYOUT_AUTO;
   s32Ret = MI_VDEC_SetOutputPortLayoutMode(eBufTileMode) ;
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_SetOutputPortLayoutMode failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }

   VdecChn = 0;
   stChnAttr.eCodecType    = E_MI_VDEC_CODEC_TYPE_H264;
   stChnAttr.u32PicWidth   = 1920;
   stChnAttr.u32PicHeight  = 1080;
   stChnAttr.eVideoMode    = E_MI_VDEC_VIDEO_MODE_FRAME;
   stChnAttr.u32BufSize    = 1024*1024;
   stChnAttr.eDpbBufMode   = E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
   stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
   stChnAttr.u32Priority = 0;

   s32Ret = MI_VDEC_CreateChn(VdecDev, VdecChn, &stChnAttr);
   if(MI_SUCCESS != s32Ret)
   {
      printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
      return s32Ret;
   }

   return MI_SUCCESS;
}
```

- Related API

N/A.

2.25. MI_VDEC_GetOutputPortLayoutMode

- Function

  Get output port layout mode

- Syntax

  MI_S32 MI_VDEC_GetOutputPortLayoutMode(MI_VDEC_DEV VdecDev, MI_VDEC_OutbufLayoutMode_e *peBufTileMode);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
peBufTileMode	Output buffer mode. Please refer to the definition given in MI_VDEC_OutbufLayoutMode_e.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID
MI_ERR_VDEC_NOT_INIT	Device not initialized.
MI_ERR_VDEC_NULL_PTR	Null pointer

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the device has been initialized; otherwise, an error code of channel not created (MI_ERR_VDEC_NOT_INIT) will be returned.

- Example

  N/A.

- Related API

  N/A.

2.26. MI_VDEC_SetDestCrop

- Function

  Set the decoded image cropping attribute

- Syntax

  MI_S32 MI_VDEC_SetDestCrop(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,

MI_VDEC_CropCfg_t *pstCropCfg);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstCropCfg	Cropping attribute. Please refer to the definition given in MI_VDEC_CropCfg_t.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_NULL_PTR	Null pointer.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Alignment Requirement: X or Width should be aligned with 16. Y or Height should be aligned with 2.
  - Before calling this function, be sure the channel to be used has been created; otherwise, an error code of channel not created (MI_ERR_VDEC_CHN_UNEXIST) will be returned.
  - If MI_VDEC_SetDestCrop and MI_VDEC_SetOutputPortAttr are calling together, make sure that the cropping size is not smaller than the scaling size. Otherwise, an error code of illegal parameter (MI_ERR_VDEC_ILLEGAL_PARAM) will be returned.

- Example

  See the example of MI_VDEC_CreateChn.

- Related API

  N/A.

2.27. MI_VDEC_GetDestCrop

- Function

  Get the decoded image cropping attribute

- Syntax

  MI_S32 MI_VDEC_GetDestCrop(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn, MI_VDEC_CropCfg_t *pstCropCfg);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstCropCfg	Cropping attribute. Please refer to the definition given in MI_VDEC_CropCfg_t.	Output

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_NULL_PTR	Null pointer.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the channel to be used has been created; otherwise, an error code of channel not created (MI_ERR_VDEC_CHN_UNEXIST) will be returned.

- Example

  N/A.

- Related API

  N/A.

2.28. MI_VDEC_ SetChnErrHandlePolicy

- Function

  Set the output strategy of the decoded channel error macroblock data frame.

- Syntax

  MI_S32 MI_VDEC_SetChnErrHandlePolicy(MI_VDEC_DEV VdecDev, MI_VDEC_CHN VdecChn,

MI_VDEC_ErrHandlePolicy_t *pstErrHandlePolicy);

- Parameter

Parameter Name	Description	Input/Output
VdecDev	Decoding device number. Range: [0, MI_VDEC_MAX_DEV_NUM).	Input
VdecChn	Video decoding channel number. Range: [0, MI_VDEC_MAX_CHN_NUM).	Input
pstErrHandlePolicy	Set the output strategy for the data frame having erroneous MB. Please refer to the definition given in MI_VDEC_ErrHandlePolicy_t.	Input

- Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_VDEC_INVALID_DEVID	Invalid device ID.
MI_ERR_VDEC_INVALID_CHNID	Invalid channel ID.
MI_ERR_VDEC_CHN_UNEXIST	Channel not created or already destroyed.
MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter.
MI_ERR_VDEC_NULL_PTR	Null pointer.
MI_ERR_VDEC_NOT_SUPPORT	The operation or function is not supported.

- Requirement
  - Header: mi_vdec.h
  - Library: libmi_vdec.a/libmi_vdec.so

- Note
  - Before calling this function, be sure the channel to be used has been created; otherwise the error code MI_ERR_VDEC_CHN_UNEXIST will be returned.

- Example

  N/A.

- Related API

  N/A.

3. DATA TYPE

The video decoder related data types and data structures are defined in the table below:

Data Structure	Description
MI_VDEC_CodecType_e	Define decoding type
MI_VDEC_DPB_BufMode_e	Define DPB buffer mode
MI_VDEC_JpegFormat_e	Define JPEG image data format
MI_VDEC_VideoMode_e	Define video stream transmission mode
MI_VDEC_ErrCode_e	Define error code type
MI_VDEC_DecodeMode_e	Define video stream decoding mode
MI_VDEC_OutputOrder_e	Define decoding output order
MI_VDEC_VideoFormat_e	Define decoded image data format
MI_VDEC_DisplayMode_e	Define display mode
MI_VDEC_OutbufLayoutMode_e	Define output buffer layout mode
MI_VDEC_InitParam_t	Define decoding device initialization parameter structure
MI_VDEC_ChnAttr_t	Define video decoding channel attribute
MI_VDEC_JpegAttr_t	Define JPEG video decoding attribute
MI_VDEC_VideoAttr_t	Define H264/H265 video decoding attribute
MI_VDEC_ChnStat_t	Define channel status structure
MI_VDEC_ChnParam_t	Define decoding channel parameter structure
MI_VDEC_VideoStream_t	Define decoding video stream structure
MI_VDEC_UserData_t	Define user data structure
MI_VDEC_OutputPortAttr_t	Define output port attribute
MI_VDEC_ErrHandlePolicy_t	Define error macroblock handle policy structure
MI_VDEC_CropCfg_t	Define cropping attribute

3.1. MI_VDEC_CodecType_e

- Description

  Define decoding type.

- Syntax

  typedef enum

```
{
    E_MI_VDEC_CODEC_TYPE_H264 = 0x0,        /* H264 */
    E_MI_VDEC_CODEC_TYPE_H265,          /* H265 */
    E_MI_VDEC_CODEC_TYPE_JPEG,          /* JPEG */
    E_MI_VDEC_CODEC_TYPE_MAX
} MI_VDEC_CodecType_e;
```

- Member

Member	Description
E_MI_VDEC_CODEC_TYPE_H264	H264 decoding.
E_MI_VDEC_CODEC_TYPE_H265	H265 decoding.
E_MI_VDEC_CODEC_TYPE_JPEG	JPEG decoding.

- Note

  Currently, users are not supported to set JPEG decoding.

- Related Data Type and Interface

  N/A.

3.2. MI_VDEC_DPB_BufMode_e

- Description

  Define DPB buffer mode.

- Syntax

```
typedef enum
{
    E_MI_VDEC_DPB_MODE_NORMAL=0,
    E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF=1,
    E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF=2,
    E_MI_VDEC_DPB_MODE_INVALID=0xFFFFFFFF
} MI_VDEC_DPB_BufMode_e;
```

- Member

Member	Description
E_MI_VDEC_DPB_MODE_NORMAL	Normal buffer mode.
E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF	Inplace one buffer mode.
E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF	Inplace two buffer mode.

- Note

  N/A.

- Related Data Type and Interface

  N/A.

3.3. MI_VDEC_JpegFormat_e

- Description

  Define JPEG image data format.

- Syntax

```
typedef enum
{
    E_MI_VDEC_JPEG_FORMAT_YCBCR400 = 0x0,  /*YUV400*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR420, /*YUV420*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR422, /*YUV 422*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR444, /*YUV 444*/
    E_MI_VDEC_JPEG_FORMAT_MAX
} MI_VDEC_JpegFormat_e;
```

- Member

  N/A.

- Note

  - The related functions of this parameter are not supported currently.

- Related Data Type and Interface

  N/A.

3.4. MI_VDEC_VideoMode_e

- Description

  Define video stream transmission mode.

- Syntax

```
typedef enum
{
    E_MI_VDEC_VIDEO_MODE_STREAM = 0x0,
    E_MI_VDEC_VIDEO_MODE_FRAME,
    E_MI_VDEC_VIDEO_MODE_MAX
} MI_VDEC_VideoMode_e;
```

- Member

Member	Description
E_MI_VIDEO_MODE_STREAM	Send video stream in stream-based method.
E_MI_VIDEO_MODE_FRAME	Send video stream in frame-based method.

- Note

  Stream-based data transmissions is currently not supported.

- Related Data Type and Interface

N/A.

3.5. MI_VDEC_ErrCode_e

- Description

  Define error code type.

- Syntax

```
typedef enum
{
    E_MI_VDEC_ERR_CODE_NONE = 0x0,
    E_MI_VDEC_ERR_CODE_UNKNOWN,
    E_MI_VDEC_ERR_CODE_MB_ERROR,
    E_MI_VDEC_ERR_CODE_REF_FRAME_ERROR,
    E_MI_VDEC_ERR_CODE_REF_FRAME_BUFF_NOT_ENOUGH,
    E_MI_VDEC_ERR_CODE_VCL_NOT_FOUND,
    E_MI_VDEC_ERR_CODE_OVER_PROFILE,
    E_MI_VDEC_ERR_CODE_OVER_LEVEL,
    E_MI_VDEC_ERR_CODE_OVER_MULTISLICE_NUM,
    E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS,
    E_MI_VDEC_ERR_CODE_FRMRATE_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_DEC_TIMEOUT,
    E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY,
    E_MI_VDEC_ERR_CODE_CODEC_TYPE_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_ERR_SPS_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_ERR_PPS_UNSUPPORT,
    E_MI_VDEC_ERR_CODE_REF_LIST_ERR,
    E_MI_VDEC_ERR_CODE_MAX
} MI_VDEC_ErrCode_e;
```

- Member

Member	Description
E_MI_VDEC_ERR_CODE_NONE	None
E_MI_VDEC_ERR_CODE_UNKNOW	Unknown error.
E_MI_VDEC_ERR_CODE_MB_ERROR	Macro block error
E_MI_VDEC_ERR_CODE_REF_FRAME_ERROR	Reference frame error
E_MI_VDEC_ERR_CODE_REF_FRAME_BUFF_NOT_ENOUGH	Insufficient number of reference frames allocated
E_MI_VDEC_ERR_CODE_VCL_NOT_FOUND	No valid image data
E_MI_VDEC_ERR_CODE_OVER_PROFILE	Profile does not support
E_MI_VDEC_ERR_CODE_OVER_LEVEL	Level does not support
E_MI_VDEC_ERR_CODE_OVER_MULTISLICE_NUM	The slices of the video frame exceed the limit

Member	Description
E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS	Illegal access, e.g. HW not yet initialized or has an error.
E_MI_VDEC_ERR_CODE_FRMRATE_UNSUPPORT	Unsupported frame rate
E_MI_VDEC_ERR_CODE_DEC_TIMEOUT	Coding timeout
E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY	Out of memory
E_MI_VDEC_ERR_CODE_CODEC_TYPE_UNSUPPORT	Unsupported decoding type
E_MI_VDEC_ERR_CODE_ERR_SPS_UNSUPPORT	Unsupported SPS or SPS error
E_MI_VDEC_ERR_CODE_ERR_PPS_UNSUPPORT	Unsupported PPS or PPS error
E_MI_VDEC_ERR_CODE_REF_LIST_ERR	Reference frame list error

- Note

  N/A.

- Related Data Type and Interface

  N/A.

3.6. MI_VDEC_DecodeMode_e

- Description

  Define video stream decoding mode.

- Syntax

```
typedef enum
{
    E_MI_VDEC_DECODE_MODE_ALL = 0x0,
    E_MI_VDEC_DECODE_MODE_I,
    E_MI_VDEC_DECODE_MODE_IP,
    E_MI_VDEC_DECODE_MODE_MAX
} MI_VDEC_DecodeMode_e;
```

- Member

Member	Description
E_MI_VDEC_DECODE_MODE_ALL	Decode IPB data frame
E_MI_VDEC_DECODE_MODE_I	Decode I frame only
E_MI_VDEC_DECODE_MODE_IP	Decode IP frame only (B skipped)

- Note

  - The related functions of this parameter are not supported currently.

- Related Data Type and Interface

  N/A.

3.7. MI_VDEC_OutputOrder_e

- Description

  Define decoding output order.

- Syntax

```
typedef enum
{
    E_MI_VDEC_OUTPUT_ORDER_DISPLAY = 0x0,
    E_MI_VDEC_OUTPUT_ORDER_DECODE,
    E_MI_VDEC_OUTPUT_ORDER_MAX,
} MI_VDEC_OutputOrder_e;
```

- Member

Member	Description
E_MI_VDEC_OUTPUT_ORDER_DISPLAY	Output data frame by display order
E_MI_VDEC_OUTPUT_ORDER_DECODE	Output data frame by decoding order

- Note

  The data frames are output in the display order by default, and user settings are not supported.

- Related Data Type and Interface

  N/A.

3.8. MI_VDEC_VideoFormat_e

- Description

  Define decoded image data format.

- Syntax

```
typedef enum
{
    E_MI_VDEC_VIDEO_FORMAT_TILE = 0x0,
    E_MI_VDEC_VIDEO_FORMAT_REDUCE,
    E_MI_VDEC_VIDEO_FORMAT_MAX
} MI_VDEC_VideoFormat_e;
```

- Member

Member	Description
E_MI_VDEC_VIDEO_FORMAT_TILE	TILE data format.
E_MI_VDEC_VIDEO_FORMAT_REDUCE	Data frame compressed format, to reduce memory used by the data frame.

- Note
  - The current data type does not support upper layer setting. You should return to the supported data types.

- Related Data Type and Interface
  N/A.

3.9. MI_VDEC_DisplayMode_e

- Description
  Define display mode.

- Syntax

```
typedef enum
{
    E_MI_VDEC_DISPLAY_MODE_PREVIEW = 0x0,
    E_MI_VDEC_DISPLAY_MODE_PLAYBACK,
    E_MI_VDEC_DISPLAY_MODE_MAX,
} MI_VDEC_DisplayMode_e;
```

- Member

Member	Description
E_MI_DISPLAY_MODE_PREVIEW	Preview mode. Will not refer to PTS output.
E_MI_DISPLAY_MODE_PLAYBACK	Playback mode. Will refer to PTS output.

- Note
  N/A.

- Related Data Type and Interface
  N/A.

3.10. MI_VDEC_OutbufLayoutMode_e

- Description
  Define output buffer layout mode.

- Syntax

```
typedef enum
{
   E_MI_VDEC_OUTBUF_LAYOUT_AUTO = 0x0,
   E_MI_VDEC_OUTBUF_LAYOUT_LINEAR,
   E_MI_VDEC_OUTBUF_LAYOUT_TILE,
   E_MI_VDEC_OUTBUF_LAYOUT_MAX
} MI_VDEC_OutbufLayoutMode_e;
```

➢ Member

Member	Description
E_MI_VDEC_OUTBUF_LAYOUT_AUTO	Automatic mode.
E_MI_VDEC_OUTBUF_LAYOUT_LINEAR	Linear mode.
E_MI_VDEC_OUTBUF_LAYOUT_TILE	Tile mode.

➢ Note

- The rotation function of the MI_DISP module is linked to the TILE mode of the MI_VDEC module. If the MI_DISP module needs to turn on the rotation function, the buffer sent by its front end must be in TILE format. The MI_VDEC module has two ways to output images in TILE format.
  1) When the output buffer mode is E_MI_VDEC_OUTBUF_LAYOUT_AUTO (default), the MI_DISP module and the MI_VDEC module need to be bound; at this time, the MI_DISP module turns on the rotation, and at the same time informs the MI_VDEC module through u64SidebandMsg, and then the MI_VDEC module automatically turns on the TILE mode to complete the rotate function. If the MI_DISP module turns off the rotation, the MI_VDEC module will also be notified through u64SidebandMsg, and then the MI_VDEC module will automatically turn off the TILE mode. This process does not require user processing and is automatically completed by the module.
  2) When the output buffer mode is E_MI_VDEC_OUTBUF_LAYOUT_TILE, the MI_VDEC module only outputs images in TILE format. The MI_DISP module must turn on the rotation function to display the TILE format images normally. In this mode, the MI_VDEC module and the MI_DISP module can be directly bound, or the user can obtain the output buffer of the MI_VDEC module, and then send it to the MI_DISP module.
- If MI_VDEC module needs to output LINEAR image, there are two ways.
  1) When the output buffer mode is E_MI_VDEC_OUTBUF_LAYOUT_AUTO (default), if the MI_VDEC module is bound to the MI_DISP module, the MI_DISP module will turn off the rotation and you can output the LINEAR image. In other cases, LINEAR images are output.
  2) When the output buffer mode is E_MI_VDEC_OUTBUF_LAYOUT_LINEAR, the MI_VDEC module only outputs LINEAR images.
- In TILE mode, the output buffer alignment of MI_VDEC module is as follows:

Chip	TILE output buffer alignment requirements (BYTE)
Taiyaki	128x32
Takoyaki	128x32
Tiramisu	128x64
Muffin	128x64

Therefore, when setting the output buffer width and height of the MI_VDEC module through MI_VDEC_OutputPortAttr_t, you also need to follow the TILE output buffer alignment requirements, otherwise the output image will be abnormal.
For example: TILE output buffer alignment requirement is 128x32, so does MI_VDEC_OutputPortAttr_t sets the width and height of MI_VDEC module output buffer.

- Related Data Type and Interface

  N/A.

3.11. MI_VDEC_InitParam_t

- Description

  Define decoding device initialization parameter structure.

- Syntax

```
typedef struct MI_VDEC_InitParam_s
{
   MI_U16 u16MaxWidth;
   MI_U16 u16MaxHeight;
} MI_VDEC_InitParam_t;
```

- Member

Member	Description
u16MaxWidth	The maximum bit stream width supported by the decoder. Parameter type: MI_U16
u16MaxHeight	The maximum bit stream height supported by the decoder. Parameter type: MI_U16

- Note

  - VDEC sets a set of maximum resolutions internally by default according to hardware performance. When decoding exceeds this value (such as 4Kx3K), you need to modify the width and height of the largest bit stream. If beyond the default value, the frame rate will drop.

- Related Data Type and Interface

  N/A.

3.12. MI_VDEC_ChnAttr_t

- Description

  Define decoding channel attribute.

- Syntax

```
typedef struct MI_VDEC_ChnAttr_s
{
   MI_VDEC_CodecType_e eCodecType;
   MI_U32      u32BufSize
   MI_U32      u32Priority
   MI_U32      u32PicWidth
   MI_U32      u32PicHeight
   MI_VDEC_VideoMode_e eVideoMode
   MI_VDEC_DPB_BufMode_e eDpbBufMode;
```

```
    union
    {
      MI_VDEC_JpegAttr_t stVdecJpegAttr;
      MI_VDEC_VideoAttr_t stVdecVideoAttr;
    };
} MI_VDEC_ChnAttr_t;
```

➢ Member

Member	Description
eCodecType	Decoding type enumeration value. Parameter type: MI_VDEC_CodecType_e
u32BufSize	Video stream buffer size.
u32Priority	Channel priority, parameter range is 1 ~ 255. The bigger the value, the higher the priority. Note: This function is currently not supported.
u32PicWidth	Max. width of decoded image supported by channel (unit: pixel)
u32PicHeight	Max. height of decoded image supported by channel (unit: pixel)
eVideoMode	Video stream transmission method. Note: Currently, only frame-based transmission is supported.
eDpbBufMode	DPB buffer mode. Note: eDpbBufMode= E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF is used only for video stream which has only one reference frame; eDpbBufMode= E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF is used only for video stream which has only two reference frames; eDpbBufMode= E_MI_VDEC_DPB_MODE_NORMAL is used for video stream more than two reference frames.
stVdecJpegAttr	JPEG channel related attribute, currently the related function of this parameter is not supported.
stVdecVideoAttr	Attribute of all supported channel types except JPEG.

※ Note

- The value of u32RefFrameNum will be invalid when eDpbBufMode is set to E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF or E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF.
- E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF and E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF are memory saving modes. The E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF decoder only allocates one reference frame, so it can only decode the video stream of one reference frame. In the same way, E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF only allocates two reference frames at most, so it can only decode the video stream of two reference frames. E_MI_VDEC_DPB_MODE_NORMAL is used for video stream more than two reference frames. For example, when set to E_MI_VDEC_DPB_MODE_NORMAL mode, to decode the video stream of one reference frame, the decoder needs to apply for two reference frames to decode normally, which will apply for 1 reference frame more than

E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF memory saving mode.

- When eDpbBufMode is E_MI_VDEC_DPB_MODE_NORMAL, you need to set the value of u32RefFrameNum to limit the maximum number of reference frames and avoid allocating too many frame buffers; if the value of u32RefFrameNum is less than the number of reference frames required for current decoding, the decoder may not decode or decode the abnormal image.
- If the number of video stream reference frames cannot be determined in advance, it is recommended to set E_MI_VDEC_DPB_MODE_NORMAL mode, and the maximum number of applied reference frames u32RefFrameNum is set to 16. The decoder will automatically analyze the number of reference frames required for the video stream to apply. For example, u32RefFrameNum is set to 16, and the current video stream decoding requires 3 reference frames, then the decoder will apply for 4 reference frames; on the contrary, if u32RefFrameNum is set to 2, the current decoding requires 3 reference frames, then the decoder can only apply for 2 reference frames and cannot decode normally.

➢ Related Data Type and Interface

N/A.

3.13. MI_VDEC_JpegAttr_t

➢ Description

Define JPEG decoding channel attribute.

➢ Syntax

```
typedef struct MI_VDEC_JpegAttr_s
{
    MI_VDEC_JpegFormat_e eJpegFormat;
}MI_VDEC_JpegAttr_t;
```

➢ Member

Member	Description
eJpegFormat	JPEG image storage format. Range is [0,E_MI_JPEG_FORMAT_MAX).

➢ Note

- Currently the related function of this parameter is not supported.

➢ Related Data Type and Interface

N/A.

3.14. MI_VDEC_VideoAttr_t

➢ Description

Define video decoding channel attribute.

- Syntax

```
typedef struct MI_VDEC_VideoAttr_s
{
    MI_U32 u32RefFrameNum;
    MI_VDEC_ErrHandlePolicy_t stErrHandlePolicy;
    MI_BOOL bDisableLowLatency;
}MI_VDEC_VideoAttr_t;
```

- Member

Member	Description
u32RefFrameNum	Reference frame number. Range: [0, 16], unit is frame. The number of reference frames decides the number of reference frames required for decoding and will influence the memory usage. We suggest setting an appropriate value according to each case concerned. For video stream testing, the recommended setting is 2.
stErrHandlePolicy	Set the output strategy for the data frame having erroneous MB. Please refer to the definition given in MI_VDEC_ErrHandlePolicy_t.
bDisableLowLatency	Whether to enable the B-frame decoding function, default: FALSE.

- Note
  - The value of u32RefFrameNum will be invalid when eDpbBufMode is set to E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF or E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF.
  - When eDpbBufMode is set to E_MI_VDEC_DPB_MODE_NORMAL, u32RefFrameNum must be set to limit the maximum number of reference frames. If the value of u32RefFrameNum is smaller than the number of the frame buffer that the decoder needs, the decoder may not start decoding or abnormal image may be outputted.
  - bDisableLowLatency=TRUE is to disable low latency. It is used to decode B-frames and reorder the output frames of B-frames, which will increase memory usage. The output image will be delayed.
  - If the scene does not need to decode the B-frame video stream, you need to set bDisableLowLatency to FALSE.

- Related Data Type and Interface

  N/A.

3.15. MI_VDEC_ChnStat_t

- Description

Define channel status structure.

- Syntax

```
typedef MI_VDEC_ChnStat_s
{
    MI_VDEC_CodecType_e eCodecType;
    MI_U32    u32LeftStreamBytes;
    MI_U32  u32LeftStreamFrames;
    MI_U32    u32LeftPics;
    MI_BOOL bChnStart;
    MI_U32 u32RecvStreamFrames;
    MI_U32 u32DecodeStreamFrames;
    MI_VDEC_ErrCode_e eErrCode;
} MI_VDEC_ChnStat_t;
```

- Member

Member	Description
eCodecType	Decoding type
u32LeftStreamBytes	Number of bytes to be decoded in video stream buffer.
u32LeftStreamFrames	Number of frames to be decoded in video stream buffer. –1 represents invalid setting. This parameter works only for frame-based transmission.
u32LeftPics	Number of pictures remaining in the image buffer.
bChnStart	Indicates whether decoder has been enabled to receive video stream.
u32RecvStreamFrames	Number of video stream frames received in the video stream buffer. –1 represents invalid setting. This parameter works only for frame-based transmission.
u32DecodeStreamFrames	Number of decoded frames in the video stream buffer.
eErrCode	Decoding error message.

- Note

Currently these parameters cannot be set.

- Related Data Type and Interface

N/A.

3.16. MI_VDEC_ChnParam_t

- Description

Define decoding channel parameter.

- Syntax

```
typedef struct MI_VDEC_ChnParam_s
{
    MI_VDEC_DecodeMode_e eDecMode;
    MI_VDEC_OutputOrder_e eOutputOrder;
    MI_VDEC_VideoFormat_e eVideoFormat;
} MI_VDEC_ChnParam_t;
```

- Member

Member	Description
eDecMode	Decoding mode, please refer to MI_VDEC_DecodeMode_e definition. Default is E_MI_VDEC_DECODE_MODE_ALL.
eOutputOrder	Please refer to the definition in MI_VDEC_OutputOrder_e. Default is E_MI_VDEC_OUTPUT_ORDER_DISPLAY, outputting frames by display order.
eVideoFormat	Please refer to the definition given in MI_VDEC_VideoFormat_e.

- Note

Currently these parameters cannot be set.

- Related Data Type and Interface

N/A.

3.17. MI_VDEC_VideoStream_t

- Description

Define decoding video stream structure.

- Syntax

```
typedef struct MI_VDEC_VideoStream_s
{
    MI_U8* pu8Addr;
    MI_U32 u32Len;
    MI_U64 u64PTS;
    MI_BOOL bEndOfFrame;
    MI_BOOL bEndOfStream;
}MI_VDEC_VideoStream_t;
```

- Member

Member	Description
pu8Addr	Video stream packet address.
u32Len	Video stream packet length, unit is word.
u64PTS	Video stream packet timestamp, unit is millisecond.
bEndOfFrame	End of frame indication (reserved function). Currently only frame-based transmission under frame mode is supported.
bEndOfStream	End of video stream indication. When all the video stream data

Member	Description
	frames have been transmitted, bEndOfStream must be set to TRUE.

- Note
  - Stream-based data transmissions is currently not supported. For frame-based case, an entire data frame should be transmitted for each transmission at a time.
  - If the video stream frame data contains PTS, the decoded output data will output the same PTS. When u64PTS=-2, the frame will be discarded by VDEC after decoding. It can be set as required.

- Related Data Type and Interface

  N/A.

3.18. MI_VDEC_UserData_t

- Description

  Define user data structure.

- Syntax

```
typedef struct MI_VDEC_UserData_s
{
    MI_U8* pu8Addr;
    MI_U32 u32Len;
    MI_BOOL bValid;
} MI_VDEC_UserData_t;
```

- Member

Member	Description
pu8Addr	User data virtual address.
u32Len	User data length, in byte unit.
bValid	Current data validity indication. Range: {TRUE, FALSE}. TRUE: Valid. FALSE: Invalid.

- Note

  The related function of this parameter is not supported currently.

- Related Data Type and Interface

  N/A.

3.19. MI_VDEC_OutputPortAttr_t

- Description

Define output port attribute.

- Syntax

```
typedef struct MI_VDEC_OutputPortAttr_s
{
    MI_U16 u16Width;                // Width of target image
    MI_U16 u16Height;               // Height of target image
}MI_VDEC_OutputPortAttr_t;
```

- Member

Member	Description
u16Width	Width of output image
u16Height	Height of output image

- Note
  - Scaling width and height need to be aligned by 2.
  - Scaling range[1/8, 1].
  - Scaling up is not supported.
  - If MI_VDEC_SetDestCrop and MI_VDEC_SetOutputPortAttr are calling together, make sure that the cropping size is not smaller than the scaling size. Otherwise, an error code of illegal parameter (MI_ERR_VDEC_ILLEGAL_PARAM) will be returned.

- Related Data Type and Interface

N/A.

3.20. MI_VDEC_ErrHandlePolicy_t

- Description

Define error macroblock handle policy structure.

- Syntax

```
typedef struct MI_VDEC_ErrHandlePolicy_s
{
    MI_BOOL bUseCusPolicy;          // FALSE: use default; TRUE: use customized value
    MI_U8 u8ErrMBPercentThreshold;
} MI_VDEC_ErrHandlePolicy_t;
```

- Member

Member	Description
bUseCusPolicy	Decide whether to customize the error frame output strategy.
u8ErrMBPercentThreshold	Set the ratio of the number of error macroblocks to the number of macro blocks in the whole data frame in the case where the data frame is not output. The value range is [0, 100].

- Note
  - If the value of bUseCusPolicy is FALSE, it means no customization, and the

u8ErrMBPercentThreshold will use the default value of 30 built in the system; if the value of bUseCusPolicy is TRUE, the customized u8ErrMBPercentThreshold value is used.

- The value range of u8ErrMBPercentThreshold is [0, 100]. For example, when the value is 10, it means that when the number of error macroblocks accounts for less than 10% of the total number of macroblocks in the whole frame, the output of the current frame will be displayed, otherwise (greater than or equal to 10%) it will not be displayed. When the value is 0, it means that frames which have error MB will not be displayed. When the value is 100, the current frame will be displayed no matter how many error MBs there are.
- The u8ErrMBPercentThreshold value calculates the ratio value based on the MB.

➢ Related Data Type and Interface

N/A.

3.21. MI_VDEC_CropCfg_t

➢ Description

Define cropping attribute.

➢ Syntax

```
typedef struct MI_VDEC_CropCfg_s
{
MI_BOOL bEnable; /* Crop region enable */
    MI_SYS_WindowRect_t stRect; /* Crop region */
 } MI_VDEC_CropCfg_t;
```

➢ Member

Member	Description
bEnable	Cropping indication.
stRect	Cropping window attribute. Please refer to the definition of MI_SYS_WindowRect_t in the document of mi_sys.

➢ Note

- Value ranges:
  u16X: [0, 4096-16], u16Y: [0, 4096-2], u16Width: [16, 4096], u16Height: [2, 4096].
- Alignment Requirement:
  u16X and u16Width need to be aligned by 16. u16Y and u16Height need to be aligned by 2.
- If MI_VDEC_SetDestCrop and MI_VDEC_SetOutputPortAttr are calling together, make sure that the cropping size is not smaller than the scaling size. Otherwise, an error code of illegal parameter (MI_ERR_VDEC_ILLEGAL_PARAM) will be returned.

➢ Related Data Type and Interface

N/A.

4. ERROR CODE

The video decoding error code is shown as below:

Error Code	Macro Definition	Description
0xA0082001	MI_ERR_VDEC_INVALID_DEVID	Invalid Device ID
0xA0082002	MI_ERR_VDEC_INVALID_CHNID	Invalid Channel ID
0xA0082003	MI_ERR_VDEC_ILLEGAL_PARAM	Illegal parameter or inputted parameter exceeding channel decoding capability
0xA0082004	MI_ERR_VDEC_CHN_EXIST	Channel to be created already exists
0xA0082005	MI_ERR_VDEC_CHN_UNEXIST	Channel does not exist
0xA0082006	MI_ERR_VDEC_NULL_PTR	Null pointer
0xA0082007	MI_ERR_VDEC_NOT_CONFIG	Not configured before use
0xA0082008	MI_ERR_VDEC_NOT_SUPPORT	This operation or function is not supported
0xA0082009	MI_ERR_VDEC_NOT_PERM	Illegal operation, for example, no user image is allowed to be inserted before performing this operation
0xA008200C	MI_ERR_VDEC_NOMEM	Memory allocation fails (due to for example insufficient memory)
0xA008200D	MI_ERR_VDEC_NOBUF	Buffer allocation fails(For example, the requested data buffer is too large)
0xA008200E	MI_ERR_VDEC_BUF_EMPTY	No data in buffer
0xA008200F	MI_ERR_VDEC_BUF_FULL	Buffer full
0xA0082010	MI_ERR_VDEC_SYS_NOTREADY	The system is not initialized or the dependent modules are not loaded
0xA0082011	MI_ERR_VDEC_BADADDR	Address error
0xA0082012	MI_ERR_VDEC_BUSY	System busy
0xA0082013	MI_ERR_VDEC_CHN_NOT_START	Channel not started or already stopped
0xA0082014	MI_ERR_VDEC_CHN_NOT_STOP	Channel cannot be closed before video stream reception is stopped
0xA0082015	MI_ERR_VDEC_NOT_INIT	Device is not initialized
0xA0082016	MI_ERR_VDEC_INITED	Device to be initialized is already initialized
0xA0082017	MI_ERR_VDEC_NOT_ENABLE	Decoder is not enabled
0xA0082018	MI_ERR_VDEC_NOT_DISABLE	Decoder is not disabled

Error Code	Macro Definition	Description
0xA008201F	MI_ERR_VDEC_FAILED	Unsuccessful

5. PROCFS INTRODUCTION

5.1. cat

- Debug Information

  Cat vdec device 0:

  # cat /proc/mi_modules/mi_vdec/mi_vdec0

  Cat vdec device 1:

  # cat /proc/mi_modules/mi_vdec/mi_vdec1

```
----------------------------------DEV Info-------------------------------------------
DevID           UUID  MaxChnNum  TotEnqCnt
    0              0         32          0

----------------------------------CHN ATTR Info--------------------------------------
ChnID  CodecType  Width  Height   BufSize  VideoMode  DpbBufMode  RefFrmNum
    0       H264   1920    1080   1048576      FRAME    INPLACE1          1
    1       H265   1920    1080   1048576      FRAME    INPLACE1          1
    2       H264   1080    1920   1048576      FRAME    INPLACE1          1
    3       H265   1080    1920   1048576      FRAME    INPLACE1          1

----------------------------------CHN PARAM Info-------------------------------------
ChnID  bCusPolicy  ErrMBDropThrd  DisplayMode  bDisableLowLatency
    0           N             30      PREVIEW                   0
    1           N             30      PREVIEW                   0
    2           N             30      PREVIEW                   0
    3           N             30      PREVIEW                   0

----------------------------------Crop & Scale Info----------------------------------
ChnID  bCrop  CropX  CropY  CropW  CropH  bScale  ScaleW  ScaleH
    0      N      0      0      0      0       Y     960     540
    1      N      0      0      0      0       Y     960     540
    2      N      0      0      0      0       Y     960     540
    3      N      0      0      0      0       Y     960     540

----------------------------------Decode Frame Info----------------------------------
ChnID  DecW  DecH  DispW  DispH  RefRecFrmCnt  LinearFrmCnt
    0  1920  1088    960    540             2             0
    1  1920  1080    960    540             2             0
    2  1088  1920    960    540             2             0
    3  1080  1920    960    540             2             0

----------------------------------Output Frame Info----------------------------------
ChnID  bTileMode  TileFormat  Width  Height  Stride  PixelFmt
    0          N          NA    960     540     960  YUV420SP
    1          N          NA    960     540     960  YUV420SP
    2          N          NA    960     540     960  YUV420SP
    3          N          NA    960     540     960  YUV420SP

----------------------------------CHN STATE------------------------------------------
ChnID  bStart  DecState      SendCnt  SendStrmSize  LeftStrmBytes     LeftCnt    DecFrmCnt    fps
    0       Y         5          124       2030919              0           0          121     25
    1       Y         5          126        981185              0           0          122     26
    2       Y         5          123       2627031              0           0          118     25
    3       Y         5          125       1282318              0           0          117     25

ChnID      Start        Get      GetOK       Done       Drop  DropNoVCL        Run  SeqChange
    0        123        123        123        121          0          2        230          0
    1        124        124        124        122          0          2        170          0
    2        121        121        121        118          0          3        192          0
    3        123        123        123        117          0          6        227          0

ChnID      Issue   Complete    InitSeq  VlcInsuff    FrmDone ChkInitSeq ChkVlcInsuff ChkFrmDone   RcvRstCB   ChkRstCB
    0          1          1          1          1        123          1            1        123          0          0
    1          2          2          2          0        124          2            0        124          0          0
    2          2          2          2          1        121          2            1        121          0          0
    3          2          2          2          0        123          2            0        123          0          0

----------------------------------ISR STATE------------------------------------------
      IsrCnt     InitSeq   VlcInsuff     FrmDone    UnderRun   HwTimeout       Other
         500           7           2         491           0           0           0
```

- Debug Information Analysis

  The printing is divided into two parts, separated by Private Vdec0 Info. The upper part is common information, and the lower part is vdec module information. It mainly records the usage and configuration attributes of the decoding channel, which can be used to check the attribute configuration and the working status of the current channel to facilitate debugging.

- Parameter Description

Parameter		**Description**
DEV Info	DevID	Hardware device number
	UUID	UUID
	MaxChnNum	The maximum number of channels
	TotEnqCnt	The number of frames queued in the decoding hardware queue.
CHN ATTR Info	DevID	Hardware device number
	ChnID	Channel ID
	CodecType	Decoding protocol type 0: H264; 1: H265;
	Width	The maximum width of the decoded image.
	Height	The maximum height of the decoded image.
	BufSize	VDEC video stream buffer size, unit: byte.
	VideoMode	Video stream send mode. FRAME: Send by frame.
	DpbBufMode	Decoding mode INPLACE1: DBP_MODE_INPLACE_ONE_BUF; INPLACE2: DBP_MODE_INPLACE_TWO_BUF; NORMAL: DBP_MODE_NORMAL;
	RefFrmNum	The maximum number of allocated reference frames.
CHN PARAM Info	DevID	Hardware device number
	ChnID	Channel ID
	bCusPolicy	Whether customers set the drop frame policy Y: Set by customers N: Not set by customers, use default
	ErrMBDropThrd	When the drop frame policy is in effect, the ratio of error macroblocks to the entire data frame.
	DisplayMode	PREVIEW PLAYBACK

Parameter		Description
	bDisableLowLatency	Whether the channel supports B-frame play. 0: Not support B-frame play. 1: Support B-frame play.
Crop & Scale Info	DevID	Hardware device number。
	ChnID	Channel ID
	bCrop	Whether to set crop function. N: Not set Y: Set.
	CropX	The starting abscissa of crop area.
	CropY	The starting ordinate of crop area.
	CropW	The width of crop area.
	CropH	The height of crop area.
	bScale	Whether to set scale function. N: Not set, output according to the size of the source video stream. Y: Set.
	ScaleW	Set the width of the scale output.
	ScaleH	Set the height of the scale output.
Decode Frame Info	DevID	Hardware device number
	ChnID	Channel ID
	DecW	The width of source video stream
	DecH	The height of source video stream.
	DispW	The width of the output data frame.
	DispH	The height of the output data frame.
	RefRecFrmCnt	The count of reference frames allocated for decoding of the current source video stream.
	linearFrmCnt	The number of Output Buffer allocated.
Output Frame Info	DevID	Hardware device number
	ChnID	Channel ID
	bTileMode	Whether to open tilemode output format. Y: Yes N: No
	TileFormat	The alignment format after opening TileMode. 128x64: 128x64 Byte alignment. 128x32: 128x32 Byte alignment. 32x32: 32x32 Byte alignment. Invalid: Not support.

Parameter		Description
		NA: Not open tilemode output format.
	Width	The width of the output data frame.
	Height	The height of the output data frame.
	Stride	The output data frame needs to be aligned width.
	PixelFmt	The pixel format of the output data frame. YUV420SP: NV12 data frame format output.
CHN STATE	DevID	Hardware device number
	ChnID	Channel ID
	bStart	Whether to start the decoder. Y: Start, enter the decoding state N: Stop the decoding state.
	DecState	The decoding state of the current channel. 0: Undefined. 1: The channel has just been created. 2: Wait for key data frames such as sps and pps. 3: Obtain information completion state of key sps, pps. 4: Before decoding, apply for the memory state of the reference frame and so on 5: Apply for the memory required for decoding under normal decoding state. 6: The channel is closed.
	SendCnt	The count of video streams sent by the application layer.
	SendStrmSize	The size of video streams sent by the application layer.
	LeftStrmBytes	The bytes to be decoded in the stream buffer.
	LeftCnt	Corresponding to SendCntParameter, the count to be decoded.
	DecFrmCnt	The count of successfully decoded output frames.
	fps	The current decoding frame rate.
CHN STATE	DevID	Hardware device number
	ChnID	Channel ID
	Start	The count to start decoding.
	Get	The count to get decoding result.
	GetOK	The count the decoding result was successfully got.
	Done	The count of successful decoding and output

Parameter		Description
		display.
	Drop	The count that the display cannot be output is dropped.
	DropNoVCL	The count that invalid data frames are dropped.
	Run	The count to apply for hardware decoding.
	SeqChange	The count the image sequence has changed. For example: SPS.
CHN STATE	DevID	Hardware device number
	ChnID	Channel ID
	Issue	The frequency to apply for hardware decoding.
	Complete	The frequency that the decoder completes decoding the video stream header information corresponding to an Issue request.
	InitSeq	The interrupt frequency of decoding video stream head information returned from decoder corresponding to an Issue request.
	VlcInsuff	The interrupt frequency of insufficient VLC Buffer.
	FrmDone	The interrupt frequency of decoding done returned from decoder.
	ChkInitSeq	Corresponding to InitSeq, the interrupt frequency checked by VDEC after the decoder completes decoding the video stream header information and returns interrupts.
	ChkFrmDone	Corresponding to FrmDone, the interrupt frequency checked by VDEC after the decoder returns decoding success and interrupts.
	RcvRstCB	The interrupt frequency of reset received from the decoder.
	ChkRstCB	Corresponding to RcvRstCB, the interrupt frequency of reset checked by VDEC.
ISR STATE	IsrCnt	The count of interrupt frequency from decoder.
	InitSeq	The interrupt frequency of decoding video stream head information reported by decoder.
	VlcInsuff	The interrupt frequency of insufficient VLC Buffer reported by decoder.
	FrmDone	The interrupt frequency of decoding done reported by decoder.
	UnderRun	The frequency of under run task reported by

Parameter		Description
		decoder.
	HwTimeout	The frequency of hardware time out reported by decoder.
	Other	The other interrupt frequency reported by decoder.

1.1. echo

vdec device 0:
/proc/mi_modules/mi_vdec/mi_vdec0
vdec device 1:
/proc/mi_modules/mi_vdec/mi_vdec1

Function	Open the debug log of capture or refresh to /proc.kmsg.
Command	echo log [capture/refresh] [on/off] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	capture/refresh
	on/off
Example	echo log refresh on > /proc/mi_modules/mi_vdec/mi_vdec0 Open the debug log of refresh to /proc.kmsg.

Function	Dump the input bitstream data from the specified channel to the specified path.
Command	echo dumpbs [chn] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	chn: Channel ID [0~63]
	path: The output path of dump file. Entering off means stop dumping bitstream.
Example	echo dumpbs 3 /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0 Dump the bitstream buffer data from channel 3 to /mnt/dump.

Function	Dump all data of the current video stream buffer from the specified channel to the specified path. (The entire video stream buffer contains valid data and invalid data, pay attention to the difference with the previous dumpbs)
Command	echo dumpbsb [chn] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	chn: Channel ID [0~63]
	path: The output path of dump file.
Example	echo dumpbsb 3 /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0

	Dump the entire video stream buffer data from channel 3 to /mnt/dump.

Function	Dump the decoded frames from the specified channel to the specified path.
Command	echo dumpfb [chn] [path] [frmcnt] [bDumpAll] [bDetile] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	chn: Channel ID [0~63]
	path: The output path of dumped file.
	Frmcnt: dumped frames
	bDumpAll: 0: Only dump yuv buffer. 1: Dump Y buffer, UV buffer and YUV buffer at the same time.
	bDetile: 0: Do not detile the YUV to be dumped (Tile format data is dumped in tilemode mode). 1: Detile the YUV to be dumped (YUV format data is dumped in tilemode mode).
Example	Channel 3, the storage path is /mnt/dump/, dumps 99 frames, does not save Y buffer and UV buffer separately, and detiles the YUV to be stored echo dumpfb 3 /mnt/dump/ 99 0 1 > /proc/mi_modules/mi_vdec/mi_vdec0

Function	Turn on/off flow checkpoint to check the running status of the decoding task. (Not recommended for customers, internal debugging)
Command	echo flowdbg [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	Status: on / off
Example	echo flowdbg on > /proc/mi_modules/mi_vdec/mi_vdec0 Turn on vdec flow checkpoint, and then enter cat /proc/mi_modules/mi_vdec/mi_vdec0 to view the checkpoints of multiple functions in vdec procfs.

Function	Turn on/off the frame drop switch, discard all decoded output data frames, and do not output to the decoded back end.
Command	echo dropoutbuf [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	Status: on / off
Example	echo dropoutbuf on > /proc/mi_modules/mi_vdec/mi_vdec0 Drop the frame buffer to be output.

Function	Turn on/off the vdec performance statistics switch, which will output more detailed proc debug information. (Not recommended for customers, internal debugging)
Command	echo setperf [Status] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	Status: on / off
Example	echo setperf on > /proc/mi_modules/mi_vdec/mi_vdec0 Turn on the vdec performance statistics switch, and then enter cat /proc/mi_modules/mi_vdec/mi_vdec0 to view the current decoding time and other data in vdec procfs.

Function	Check the time interval consumed by each state of the decoding process. (Not recommended for customers, internal debugging)
Command	echo flowstat [chn] [on/off] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	chn: Channel ID [0~63]
	on/off: on / off
Example	echo flowstat 3 on > /proc/mi_modules/mi_vdec/mi_vdec0 Turn on channel 3 to check that each state of the decoding process consumes time interval.

Function	Debug inside the refresh function. (Not open to customers, internal platform debugging.)
Command	echo refresh [chn] [pattern/dumpref/md5/clroutbuf] [on/off] [path] > /proc/mi_modules/mi_vdec/mi_vdec0
Parameter Description	chn: Channel ID [0~63]
	pattern/dumpref/md5/clroutbuf: pattern: Do not use the reference frame of the decoder, use the local file to decode and output the NV12 data frame. Dumpref: The storage decoder is used to decode the reference frame of the output NV12. md5: Check whether all md5 values of the entire process of refresh are consistent, and check that the memory problem debug is turned on. clroutbuf: Clear the SCL output buffer data before transcoding.
	on/off: on / off
	path: The path of local file and dumped file.
Example	echo refresh 3 md5 on /mnt/dump/ > /proc/mi_modules/mi_vdec/mi_vdec0 Turn on channel 3 to check the value of md5 of refresh.